

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence
NAME: PREETHI R

COURSE CODE: CSA1710
REG NO:192511352

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

Duration : 1 Hour
Total Marks:50

Design Task

Code of Conduct:

*I _PREETHI R (192511352)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

preethir

Design Task

AI-Based Network Intrusion Detection Framework

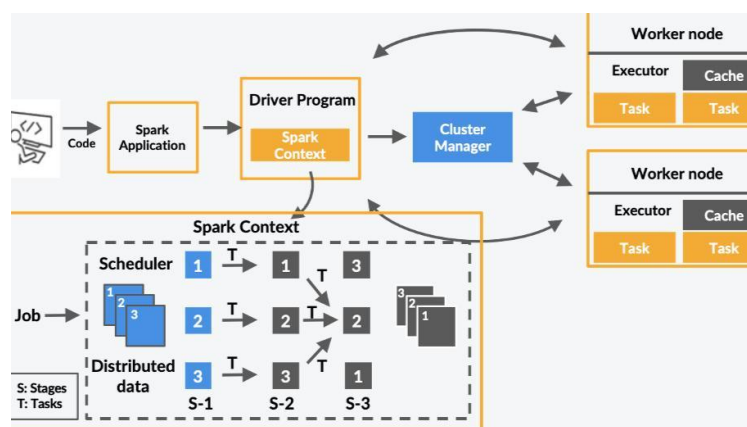
Design a scalable intrusion detection system using PySpark RDDs to process large volumes of network traffic logs in real time. Explain how anomaly detection algorithms can be implemented in a distributed manner across a cluster, and how intelligent agents collaborate to identify suspicious traffic patterns, trigger security alerts, and continuously update detection rules based on newly observed attack signatures.

AI-Based Network Intrusion Detection Framework

1. Introduction

Network security has become increasingly important because modern networks generate huge amounts of traffic every second. Traditional intrusion detection systems may face difficulties when processing such large volumes of data. An AI-Based Network Intrusion Detection Framework combines Artificial Intelligence, distributed computing, and intelligent agents to identify malicious network activities efficiently.

The proposed framework uses PySpark RDDs to distribute network traffic logs across multiple computing nodes. AI-based anomaly detection algorithms analyze the traffic and identify unusual patterns. Intelligent agents then collaborate to verify suspicious activities, generate security alerts, and update detection rules when new attack patterns are discovered.



Objectives

The main objectives of the proposed system are:

- To process large volumes of network traffic using PySpark RDDs.
- To distribute intrusion detection tasks across multiple cluster nodes.
- To identify abnormal and malicious network activities using AI algorithms.
- To enable intelligent agents to collaborate during threat detection.
- To generate security alerts based on the severity of detected threats.
- To continuously improve detection rules using newly identified attack signatures.
- To provide a scalable and adaptive intrusion detection solution.

Proposed System

The proposed system consists of several stages. Network traffic logs are first collected from different sources such as servers, routers, firewalls, and network monitoring devices. These logs are converted into PySpark RDDs and distributed across the cluster.

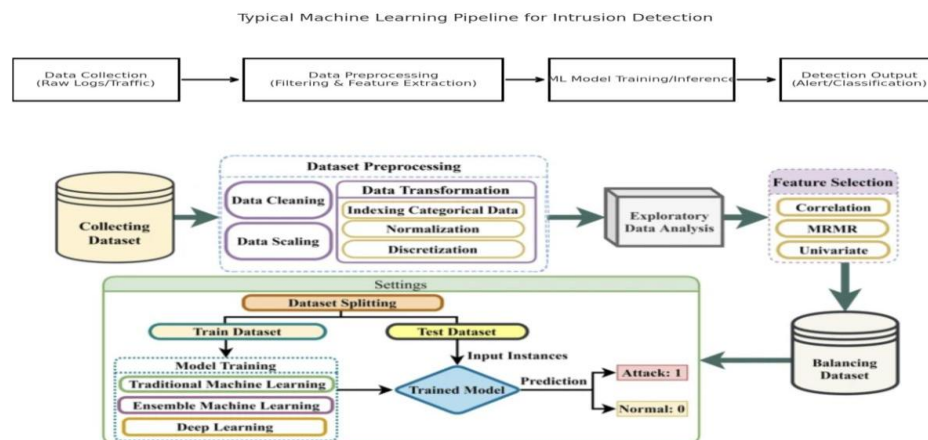
Each worker node processes its assigned portion of the traffic. Important features such as source IP, destination IP, port number, protocol, packet size, connection duration, and packet frequency are extracted.

The extracted features are then provided to AI-based anomaly detection algorithms. Suspicious traffic is forwarded to intelligent agents for further analysis and threat correlation.

4. System Architecture

The overall architecture consists of the following components:

Network Traffic → Data Collection → PySpark RDD → Distributed Processing → Feature Extraction → AI Anomaly Detection → Intelligent Agents → Threat Analysis → Security Alert → Rule Update



5. PySpark RDD-Based Processing

PySpark RDDs are used to distribute network traffic logs across multiple worker nodes. An RDD is a distributed collection of data that can be processed in parallel.

A simplified implementation is:

```
logs = sc.textFile("network_logs.csv")
```

```
features = logs.map(lambda x: extract_features(x))
```

```
suspicious = features.filter(  
    lambda x: anomaly_score(x) > THRESHOLD  
)
```

Operations such as `map()`, `filter()`, `reduceByKey()`, and `groupByKey()` allow large network datasets to be processed efficiently.

For example, millions of network records can be divided into different partitions and processed simultaneously by several worker nodes.

6. Distributed Anomaly Detection

Anomaly detection identifies traffic that differs significantly from normal network behavior.

The system can use different techniques:

Statistical Analysis:

Normal values such as average packet rate, connection duration, and traffic volume are calculated. Significant deviations are treated as anomalies.

Clustering:

Traffic records are grouped according to similar characteristics. Records that do not fit established groups can be identified as suspicious.

Machine Learning Classification:

Machine learning models can classify traffic as normal or malicious based on previously trained network traffic datasets.

Possible attacks detected include:

- Denial-of-Service attacks
- Distributed Denial-of-Service attacks
- Port scanning
- Brute-force attacks
- Malware traffic
- Suspicious login attempts

7. Intelligent Agent Collaboration

The system uses multiple intelligent agents, where each agent performs a specific security task.

Intelligent Agent	Function
Traffic Monitoring Agent	Monitors incoming network traffic
Anomaly Detection Agent	Identifies unusual traffic patterns
Signature Agent	Compares traffic with known attack signatures
Correlation Agent	Combines results from multiple agents
Alert Agent	Generates and prioritizes security alerts
Learning Agent	Updates detection rules with new attack patterns

For example, when an unusually large number of requests originates from a particular source, the Anomaly Detection Agent identifies the abnormal behavior. The Signature Agent checks whether the pattern matches a known attack. The Correlation Agent combines these results and determines the final threat level.

8. Threat Detection and Security Alerts

After analyzing the traffic, the system assigns a threat score based on the detected characteristics.

Threat Score	Risk Level	Response
0–30	Normal	Continue monitoring
31–60	Low	Record the event
61–80	Medium	Investigate traffic
81–100	Critical	Generate immediate alert

ALERT: Possible DDoS Attack

Source IP: 192.168.x.x

Destination Port: 80

Threat Level: CRITICAL

Recommended Action: Investigate / Block

9. Continuous Rule Updating

One of the important features of the proposed framework is continuous learning.

When an unknown traffic pattern is detected, the system analyzes the behavior using multiple agents. If the pattern is confirmed as malicious, the Learning Agent creates or updates an attack signature.

The process is:

New Traffic → Anomaly Detected → Agent Investigation → Attack Confirmed → New Signature → Rule Repository Updated → Future Traffic Detection

This allows the system to adapt to newly emerging attack patterns instead of depending entirely on previously defined rules.

10. Scalability

The use of PySpark allows the system to scale according to the amount of network traffic.

For example, if a network generates millions of records, the records can be divided into several RDD partitions.

Large Network Dataset → RDD Partitions → Worker Node 1 + Worker Node 2 + Worker Node 3 + Worker Node 4 → Parallel Processing → Combined Results

Additional worker nodes can be added when the traffic volume increases. Therefore, the framework is suitable for large organizations, cloud environments, and enterprise networks.

11. Advantages

- Handles large-scale network traffic.
- Provides distributed and parallel processing.
- Reduces processing time.
- Detects both known and unusual traffic patterns.
- Supports intelligent multi-agent collaboration.
- Generates automated security alerts.
- Continuously updates detection rules.
- Can be scaled by adding cluster nodes.
- Improves adaptability against emerging attacks.

12. Conclusion

The AI-Based Network Intrusion Detection Framework provides a scalable and intelligent approach to network security. PySpark RDDs enable large volumes of network traffic to be processed in parallel across a cluster. AI-based anomaly detection identifies suspicious behavior, while intelligent agents collaborate to verify threats and generate security alerts.

The Learning Agent further improves the system by updating detection rules based on newly discovered attack signatures. Therefore, the proposed framework can provide a scalable, adaptive, and intelligent solution for modern network intrusion detection.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations /actions	Mostly correct usage	Limited or partial use	Incorrect/missing

Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation